

Fixed-Point Bit-Width Study

Choosing the arithmetic widths for a hardware triangle rasterizer by
measuring the software one

Rasterizer_Simulator

Contents

Fixed-Point Bit-Width Study	2
Summary	2
Results	2
Scope	2
1. The objection, answered first	3
2. Method	3
The instrument	3
Signals	4
Configuration	4
Instrument construction	4
3. Integer fields: the histogram study	5
3.1 The edge accumulator	5
3.2 What narrowing costs	6
3.3 area — the check that makes the rest meaningful	7
3.4 delta — the multiplier inputs, and how loose the textbook rule is	7
3.5 screen — a tripwire, not a discovery	8
3.6 recw / invw — the one bound that cannot be derived	8
3.7 bary — theory to four digits, and the bug it caught	9
3.8 z — the integer field is empty	10
3.9 zdiff — the depth answer, read bottom-up	10
3.10 Resolution scaling	12
4. The fractional field: the sub-pixel sweep	12
Architecture	12
The metric: read diff ≥ 2	13
Result	13
Convergence at $s = 16$ — the correctness proof	14
The noise floor scales inversely with tessellation density	15
5. Conclusion	15
Architectural findings	16
Acceptance criteria for the hardware implementation	16
6. What transfers, and what does not	17
7. Limitations	17
8. Reproduction	17
Data	18
Appendix — signal summary, all runs	18

Consistency checks	19
Miscellaneous counters	19

Fixed-Point Bit-Width Study

Choosing the arithmetic widths for an RTL triangle rasterizer, by measuring the software one.

Summary

An RTL rasterizer has no floats. Every signal is declared `reg [N-1:0]`, and every `N` must be chosen. Too small and values wrap — a wrapped edge function inverts its sign, the coverage test inverts with it, and the triangle renders inside-out. Too large and area is wasted, since multiplier cost scales roughly as N^2 .

This study instruments the software rasterizer to choose those widths from measurement rather than from worst-case analysis, and to establish the pixel difference expected between the two implementations, so that a hardware-versus-software comparison can be judged.

It has two parts: an exponent histogram over nine signals sizes the **integer** fields, and a configurable fixed-point coverage path swept against the float reference sizes the **fractional** field.

Results

quantity	measured	from analysis alone
Edge accumulator, integer field	19 bits	22 (geometric bound), 23 (Q propagation)
Sub-pixel fractional bits <code>s</code>	4	not derivable
Edge accumulator total, $19 + 2s$	27 bits	31
Depth field, integer part	0 bits	1
Depth field, fractional part	16 bits	not derivable
Divider denominator range	[0.25, 2)	unbounded

Two results that could not have been derived:

- Required width is **uncorrelated with triangle count**. Iron Man has $17\times$ the torus knot's triangles and needs **two more bits**, not fewer.
- The one-level noise floor between the float and fixed paths scales **inversely** with tessellation density, and saturates.

Scope

Float everywhere up to and including the viewport transform; fixed-point from `screenVertex` onward.

That boundary is not a simplification — it is where real GPUs draw the same line. Vertex processing is IEEE float because the MVP chain has unbounded dynamic range. Rasterization is fixed-point because coverage and depth must be *exact and reproducible*, so two triangles sharing an edge compute bit-identical values for it and no crack appears between them.

1. The objection, answered first

“19 bits is just what your model happened to need. Load a different OBJ and get a different number.”

Correct. 19 is a property of `solids_scene.obj`, not of rasterization. Any framing that presents it as *the answer* is dressing a measurement up as a discovery.

The number was never the output. This is:

$$W_{\text{edge}} = 2 \log_2 L + 2$$

where **L is the largest triangle’s screen extent in pixels**. That relation is mesh-independent and was confirmed by two independent mechanisms — three meshes varying L through triangle size, and three resolutions varying it through pixel scale.

And L is a hardware parameter, not an input. If the geometry front end guarantees no triangle exceeds a chosen screen extent — splitting any that do — the width follows from a design decision rather than from whatever mesh someone loads. That turns “19 works for my three test meshes” into “W works for any mesh, given the splitter.”

Three positions are defensible, and they differ in *where* the cost is spent:

choice	guarantee	cost
22-bit integer field, no splitter	any mesh, unconditional	1.34× multiplier area
19-bit integer field, no splitter	only meshes like the three tested	breaks on a skybox quad
19-bit field + triangle splitter	any mesh, unconditional	splitter logic in the geometry stage

The middle row is the indefensible one. **The recommendation for a first hardware implementation is the top row** — 22 integer bits, $22 + 2s = 30$ total. A splitter is a whole feature (size test, subdivision, changed triangle count downstream, knock-on effects on the command interface) and introduces a second problem before the first is solved. Three bits of multiplier on an FPGA is close to free: DSP slices, not wafer area.

What the study is then for, given that the safe choice was available all along:

- Knowing the safe choice costs 1.34× the minimum, **because the minimum was measured**. That is a different statement from “22 because the bound said so.”
- Knowing where the splitter threshold goes, if those bits are ever wanted back.
- The findings that do not depend on the mesh (§6), which change how the *next* block gets sized.
- The narrowing cost curves (§3.2), which are a design space rather than a number, and let someone else make a different choice from the same data.

2. Method

The instrument

For each instrumented value the code records `std::ilogb(v)`, the integer e with $2^e \leq |v| < 2^{(e+1)}$. That is the index of the most significant set bit, so a value binned at e needs

$W = e + 2$ bits ($e + 1$ magnitude, plus a sign)

and **the histogram is the bit-width distribution, already computed**. Reading the cumulative sum from the top gives “W bits covers X% of all evaluations.”

ilogb reads the float’s exponent field directly. It is not a logarithm and costs one instruction.

Why a histogram and not a min/max pair. A maximum is one point; a histogram is a design space. The maximum says 19 bits. The histogram says 18 bits breaks 1.0% of evaluations, 17 breaks 5.8%, 16 breaks 14.2% — and that turns a number into an engineering decision.

Signals

signal	tallied where	sizes
edge	raw w0/w1/w2, per bounding-box pixel	coverage accumulator
area	signed triangle area, per triangle, pre-cull	barycentric denominator
delta screen	six edge deltas, per triangle vertex x,y after viewport transform	multiplier inputs vertex register
recw	vertex rec_w	reciprocal conditioning
invw	interpolated inv_w_pixel	divider denominator
bary	normalised w0..w2	verification, and the sweep bound
z	interpolated NDC depth, per covered fragment	depth field range
zdiff	$\backslash z_{\text{new}} - z_{\text{buffer}} $ at the test, before the branch	depth fractional bits

Configuration

120 frames for the histogram study, 10 for the sweep; 3°/frame, camera translate −2 on Z, 60° vertical FOV, near 0.1, far 100. Models normalised to span 2 units on their largest axis and centred at the origin. Three meshes at 1080p, plus the solids scene at 720p and 480p.

mesh	vertices	triangles	character
solids_scene	1,968	3,636	six flat-faced solids + smooth torus, interpenetrating
torus_knot	6,400	12,800	uniform procedural sweep, heavy self-occlusion
IronMan	129,759	217,038	human-modelled: large panels and dense regions

Instrument construction

Details that cost real debugging time and are worth recording:

- **Compile-time switched.** One #define in stats.h; TALLY(sig, v) expands to ((void)sizeof(sig), (void)sizeof(v)) when off. No code is emitted, but the operands are still *named*, so a tally-only variable does not trip -Wunused-variable under -Werror, and it remains an expression so it parses in any statement position.

- **Call sites carry no #if.** An earlier version guarded each call site directly; a misspelled variable survived three review passes because the preprocessor deleted the line before the compiler saw it. Moving the #if into the header removed that failure mode permanently.
- **The switch lives in a header, not the Makefile.** A -D flag changes no file’s modification time, so make hands back a stale object.
- **uint64_t counters, not long.** MinGW is LLP64, so long is 32 bits. The Iron Man run recorded **2,136,285,247** edge tallies against an INT32_MAX of 2,147,483,647 — within 0.5% of overflow.
- **PRiU64, not %llu.** MinGW links MSVCRT, whose printf rejects the ll length modifier. A rejected conversion also desynchronises every argument after it.
- **Guards.** `ilogb(0)` returns INT_MIN and `ilogb(inf)` returns INT_MAX; both are counted separately, along with index range checks. A nonzero `clamp_hi` is a bug signal, not a data point.

3. Integer fields: the histogram study

3.1 The edge accumulator

The edge function is evaluated three times per pixel of every triangle’s bounding box, which makes it the highest-traffic arithmetic in the design.

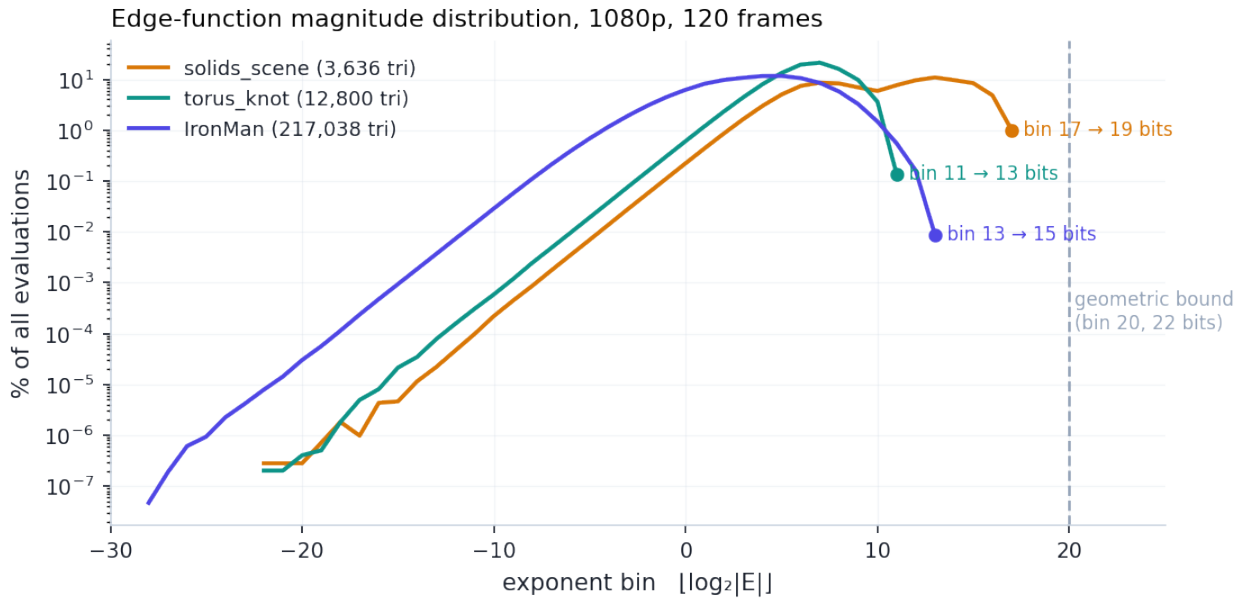


Figure 1: Edge function magnitude distribution for three meshes

mesh	tallies	top bin	W
solids_scene	715,426,158	17	19 bits
torus_knot	995,171,265	11	13 bits
IronMan	2,136,285,247	13	15 bits

The geometric bound is the largest triangle inscribable in the viewport, $E \leq 1920 \times 1080 \rightarrow$ bin 20 → **22 bits**. Measurement saves three bits, which at N^2 scaling is 34% of the block’s dominant arithmetic.

In fixed point every bin shifts by $2s$, since multiplying by a power of two adds to the exponent and leaves the mantissa alone. So $W_{\text{edge}} = 19 + 2s$.

The counterintuitive part

Width is **uncorrelated with triangle count**. Reasoning from mean triangle size predicts bin 7 for Iron Man; the answer is 13 — six bits high.

The error is reasoning from the *mean* when the top bin is set by the *maximum*. Triangle count gives the mean; it says nothing about the tail, and the tail sizes the datapath. The torus knot is swept by a parallel-transport frame, so every quad is nearly the same size and $\text{mean} \approx \text{max}$. Iron Man was modelled by a human — large flat panels on the torso and base, dense tessellation on hands and face.

Shape encodes tessellation uniformity

Visible in the figure, and interpretable without running the analysis:

- **torus_knot** — one clean lobe, counts doubling per bin (1.97, 1.94, 1.89 across bins 0→3) then a hard cutoff. The doubling is derivable: E is affine in the sample point, so its values across a bounding box are roughly uniform, and a uniform distribution binned by exponent puts $\text{count} \propto 2^e$ in bin e .
- **solids_scene** — sharply bimodal, peaks at bins 7 and 15, separated by 2^9 in area ($\approx 23\times$ in linear extent): flat-faced solids versus sphere tessellation.
- **IronMan** — a broad plateau; bins 3, 4, 5 hold 228M, 246M, 247M. A single triangle size gives one lobe; a superposition of many sizes, each with its own cutoff, sums to a plateau. The histogram is flat because the mesh contains triangles of every size.

3.2 What narrowing costs

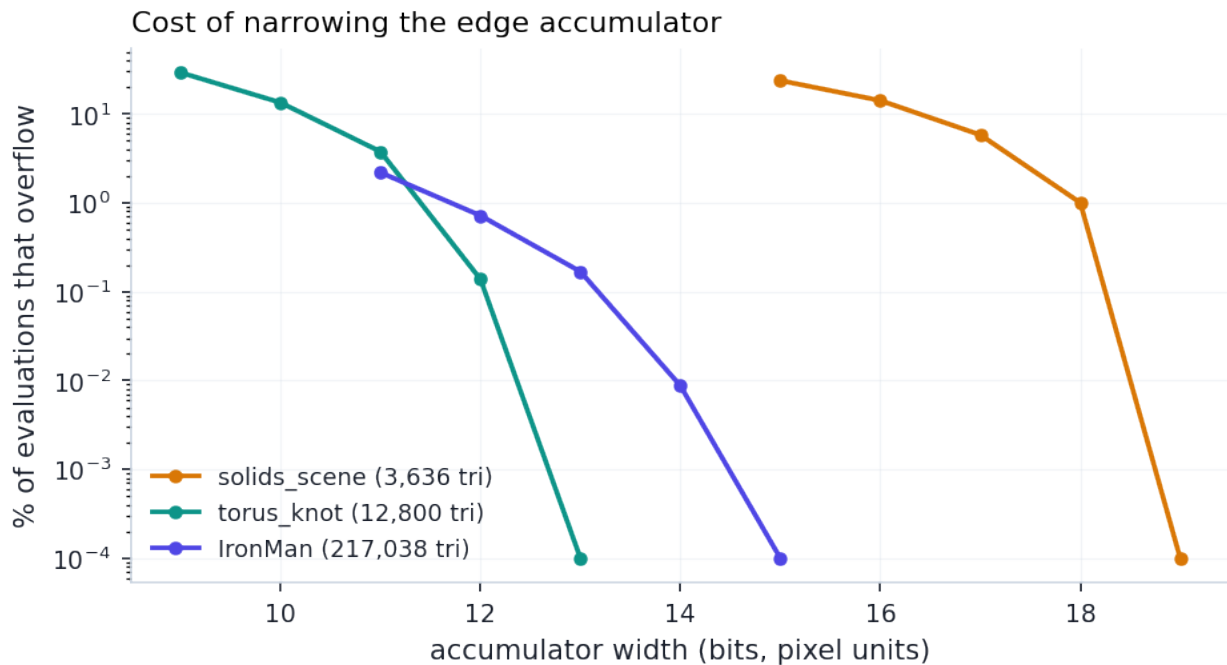


Figure 2: Fraction of evaluations that overflow at each accumulator width

mesh	W (safe)	−1 bit	−2 bits	−3 bits
solids_scene	19	1.004%	5.821%	14.172%
torus_knot	13	0.138%	3.767%	13.424%
IronMan	15	0.0087%	0.168%	0.718%

Same block, opposite conclusions per mesh. Iron Man’s top bin holds 186,646 of 2.14 billion evaluations — one in eleven thousand. At that rate, 14 bits plus a saturating path is a defensible trade. The solids scene at one in a hundred does not support it.

3.3 area — the check that makes the rest meaningful

At any sample point the three edge functions satisfy

$$w_0 + w_1 + w_2 == \text{area}$$

exactly — an algebraic identity, not an approximation; the sample coordinates cancel when the three are expanded. *Verified by hand:* $A=(1209.4481, 420.4052)$, $B=(1199.7294, 428.4380)$, $C=(1220.3883, 415.9513)$, $P=(1210.5, 421.5)$ gives $-8.842 - 16.663 - 19.090 = -44.595$ against a printed area of -44.5939 .

Inside the triangle all three share a sign, so none can exceed the total:

$$\max |E(P)| == |\text{area}|$$

reached at the vertex opposite the edge — a fixed geometric point, not a random pixel. **This answers the “isn’t it just chance?” objection:** values near zero are set by pixel-to-edge proximity and are genuinely random, populating the long low tail; values at the top are geometry and are fixed the moment the triangle is projected.

The prediction is that area’s top bin must equal edge’s. It does, in all five configurations:

	solids 1080p	torus 1080p	IronMan 1080p	solids 720p	solids 480p
area top bin	17	11	13	16	15
edge top bin	17	11	13	16	15

Not the same measurement dressed up: area was tallied 436,320 times on the solids scene against edge’s 715 million — a different code path, a different loop, 1,640× the samples, the same answer.

3.4 delta — the multiplier inputs, and how loose the textbook rule is

The rule $W_{\text{product}} = W_a + W_b$ (plus a guard bit for the subtraction) assumes both operands hit maximum simultaneously with opposite signs. Measuring the inputs and the output separately quantifies how wrong that is — no other pair of signals in the study permits checking a propagation rule against reality.

run	delta top	W_{Δ}	rule says $2W+1$	geometric $E \leq L^2$	measured	slack
solids 1080p	9	11	23	21	19	4
torus_knot 1080p	6	8	17	15	13	4
IronMan 1080p	7	9	19	17	15	4

run	delta top	W_Δ	rule says 2W+1	geometric $E \leq L^2$	measured	slack
solids 720p	8	10	21	19	18	3
solids 480p	7	9	19	17	17	2

Where the gap comes from, with every delta under L:

- The rule assumes $|E| \leq 2L^2$.
- Geometry gives $|E| = 2 \cdot \text{area} \leq L_x \cdot L_y \leq L^2$. **One bit** — the rule does not know the two products are geometrically linked rather than independent.
- The remaining 1–3 bits: the triangle carrying the largest single delta is not the one with the largest area. A long thin sliver has a big Δ and almost no area, so the rule sizes for a triangle that does not exist in the mesh.

Three widths for the same signal: 23 (Q propagation), 22 (geometric bound), 19 (measured). A 23-bit multiplier against a 19-bit one is $1.47\times$ the area.

Uncertainty on the 4. The three meshes all give exactly 4, but the resolution sweep gives 2, 3, 4 — so the figure is not a constant. The spread is instrument granularity rather than physics: $\text{slack} = 2 \cdot b_\Delta - b_E + 3$, and each bin is a floor of $\log_2$, so two bins with one of them doubled carry ± 2 bits of uncertainty. The defensible statement is **2–4 bits, consistent with a true value near 3**, and it would resolve to a single figure by tracking a running float maximum alongside the histogram.

This finding is not about the test meshes. It is about the propagation rule being pessimistic whenever a multiplier's operands are geometrically related — which recurs in every dot product, cross product and matrix multiply whose operands come from the same object.

3.5 screen — a tripwire, not a discovery

Trivially derivable: NDC spans $[-1, 1]$, so screen x spans $[0, 1920)$ and $\lfloor \log_2 1920 \rfloor = 10$, giving 11 bits unsigned.

Instrumented anyway for two reasons. It sizes the register that feeds the multiplier, one level up the dataflow from delta. And it is a bug detector: this renderer has **no near-plane clipping**, so a vertex at or behind the eye produces a garbage screen coordinate. The code survives because bounding_box clamps — but the clamp happens *after* screen_verts is written, and in RTL that register has a fixed width.

Result: top bin 10, clamp_hi = 0, in every run. Clean. The 480p run is a self-check — $854 < 1024 = 2^{10} \rightarrow$ bin 9, while 1280 and 1920 both sit in $[1024, 2048) \rightarrow$ bin 10. The instrument moved exactly where arithmetic says it should.

The absence of clamps confirms **these meshes** stay in front of the camera. It does not prove the pipeline is safe; move the camera into the model and this histogram grows a high tail immediately.

3.6 recw / invw — the one bound that cannot be derived

Every other signal has a derivable ceiling. Division does not.

The textbook rule $Qa.b \div Qc.d \rightarrow Q(a-c).(b-d)$ **is not derivable**: as $y \rightarrow 0$, $x/y \rightarrow \infty$, so no format on y bounds the quotient. The rule silently assumes $|y| \geq 1$. The honest bound needs

$$|x/y| \leq |x|_{\max} / |y|_{\min}$$

and $|y|_{\min}$ is a fact about the scene. It is not in any format, any rule, or any textbook. It only exists if measured.

signal	occupied bins	range
recw (per vertex)	-2 .. 0	[0.25, 2)
invw (per fragment)	-2 .. 0	[0.25, 2)

Three bins, identical in every run. **The denominator never goes below 0.25**, so the divider output lies in (0.5, 4] — three integer bits, derived from the measurement without further instrumentation. The existing `fabs(inv_w_pixel) < 1e-8f` guard, inserted as a placeholder for exactly this question, has never fired.

`invw` occupying exactly the same bins as `recw` is a correctness proof rather than a coincidence: `inv_w_pixel` is a *convex* combination, so it must lie between the smallest and largest of its three inputs.

Caveat. [0.25, 2) is a property of camera-at-2 with a model normalised to span 2 units. Move the camera into the model and $w \rightarrow 0$. That is the same condition that makes near-plane clipping necessary: **the clip stage and the divider bound are the same problem**, and clipping's guarantee $w \geq w_{\text{near}}$ is what makes the divider sizable at all.

3.7 bary — theory to four digits, and the bug it caught

For a uniformly sampled point in a triangle, $\{w_0 > c\}$ is a similar triangle scaled by $(1-c)$, so $P(w_0 > c) = (1-c)^2$ and the density is $f(w) = 2(1-w)$. The fraction landing in bin e is

$$\int_{2^e}^{2^{e+1}} 2(1-w) dw = 2 \cdot 2^e - 3 \cdot 2^{2e}$$

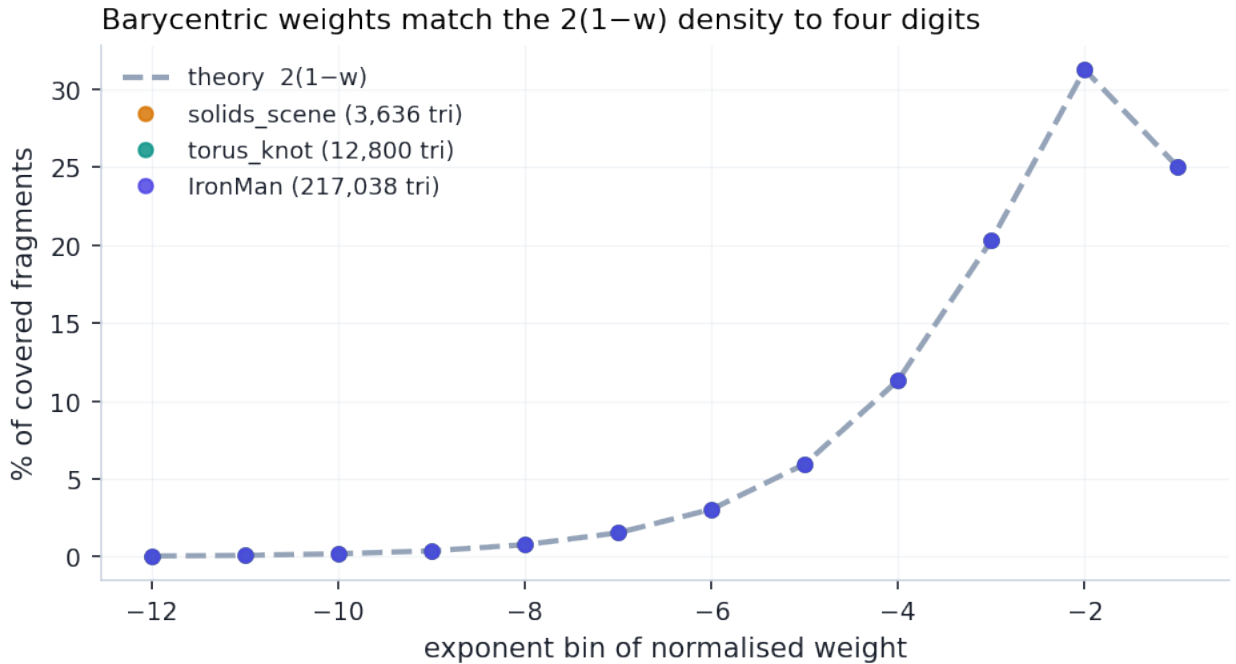


Figure 3: Barycentric weight distribution against the $2(1-w)$ prediction

Solids scene, 202,651,317 samples, **no fitted parameters**:

bin	-1	-2	-3	-4	-5	-6
predicted	0.2500	0.3125	0.2031	0.1133	0.0596	0.0305
measured	0.2500	0.3125	0.2031	0.1133	0.0596	0.0305

Barycentric coordinates are affine-invariant, so shape, size and count all cancel: a sliver and a large equilateral triangle produce identical distributions. Only the sample count scales.

Three uses:

- **Verification.** Matching to four digits proves the weights are correctly normalised and sum to 1 — stronger than any image check, since a scaling bug would show as a shifted distribution while still looking plausible on screen.
- **It bounds the sweep.** The tail reaches bin -30: fragments sitting $\sim 10^{-9}$ of a pixel from an edge. Those are precisely the fragments whose coverage flips under quantisation, so their count is the floor on how many pixels can possibly differ between the two renderers.
- **It caught a real bug.** An early z run produced this distribution on all three meshes, with ratios constant to 0.5% across eight octaves. Two mesh-specific hypotheses were pursued and neither could explain the identity; fitting the shape and finding $2(1-w)$ did. The call site was tallying w_0 instead of z . **A quantity identical across three unrelated meshes is almost never geometry** — cross-mesh comparison found this, not looking at the image.

3.8 z — the integer field is empty

NDC depth after the perspective divide is $z = A - B/d$, with $A = (f+n)/(f-n) = 1.002$ and $B = 2fn/(f-n) = 0.2002$. The key property is that z depends on $1/d$, not d .

Result: one bin, bin -1, in every run of every mesh at every resolution. Every depth value ever produced lies in $[0.5, 1)$.

A single occupied bin looks like a useless measurement. It is not — run the formula backwards:

$$z \geq 0.5 \rightarrow d \geq 0.399 \rightarrow \text{bounding radius} \leq 1.601 \rightarrow d_{\max} = 3.601 \rightarrow z_{\max} = 0.946$$

$z \in [0.500, 0.946]$, a span of 0.446 out of the available 2.0 — **22.3% of the NDC range**. The histogram measured the model's bounding sphere without being told anything about it.

The depth field needs zero integer bits. Stored as Q1.15, every fragment reads 0.1xxxxxxxxxxxxx: sign always 0, first fractional bit always 1. Two of sixteen bits carry no information, and an affine remap $(z - 0.5) \times 2$ recovers both.

Precision is non-uniform by 6.35 bits. Differentiating, $dz/dd = 0.2002/d^2$:

position	d	dz/dd
front of model	0.399	1.259
back of model	3.601	0.01544

A ratio of $81.5\times$, or $\log_2 81.5 = 6.35$ **bits**. A fixed-point depth buffer has uniform LSB spacing, so a separation resolvable at the front needs 6.35 more bits at the back. That is the $1/z$ front-loading, quantified — and it is why modern hardware uses reversed-Z with a floating-point depth buffer, whose finer spacing near zero cancels the compression almost exactly.

3.9 zdiff — the depth answer, read bottom-up

Every other signal is read top-down: the largest value sets the integer bits. This one is read **bottom-up**. The comparator's job is to tell two surfaces apart; if the smallest real separation

is 2^{-k} and the field has fewer than k fractional bits, the two quantise to the same value and z-fight.

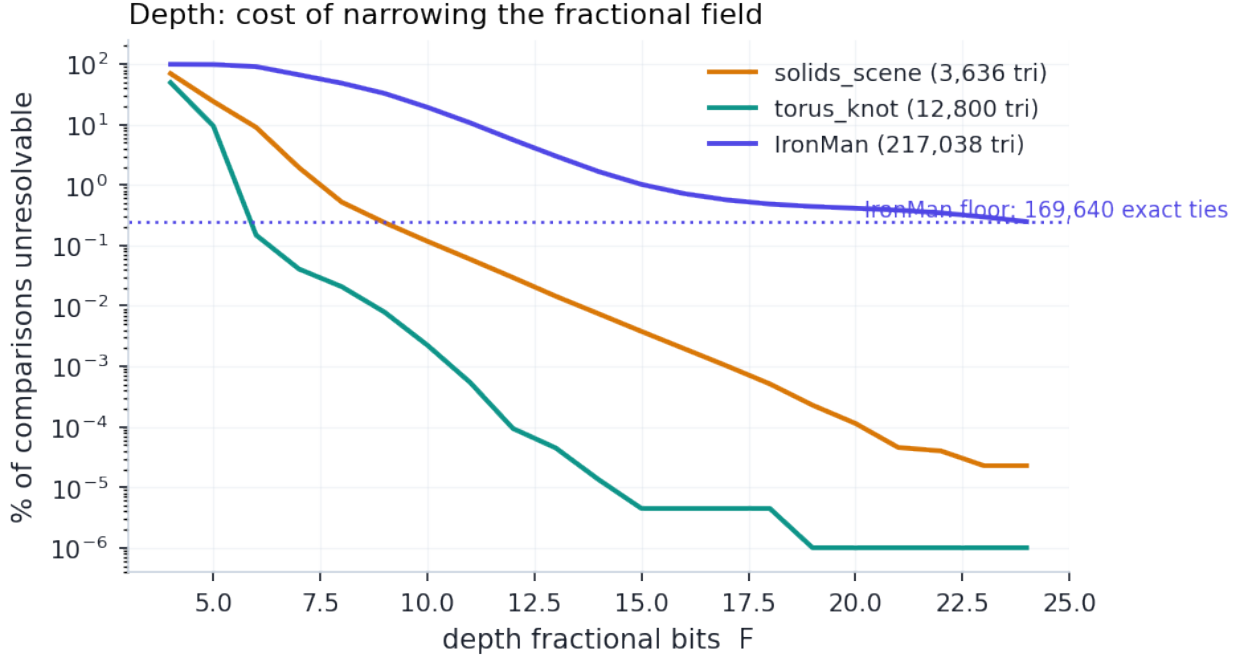


Figure 4: Fraction of depth comparisons unresolvable at each fractional width

F	solids	torus_knot	IronMan
6	8.984%	0.148%	90.750%
8	0.521%	0.021%	48.325%
12	0.030%	0.0001%	5.597%
16	0.002%	~0	0.720%
24	0.00002%	0	0.248%

The torus knot has a cliff. 0.148% at $F=6$, 9.49% at $F=5$ — a $64\times$ jump for one bit, where the raw counts drop $87\times$ in a single bin. That is the boundary between real geometry (the tube passing in front of itself) and the noise floor of near-coincident surfaces where the tube runs tangent to itself. Read the knee, not the last nonzero bin.

Iron Man does not converge. It asymptotes at 0.248% and stops, because `zdiff` recorded **169,640 exact zeros** — comparisons where two surfaces produce bit-identical NDC depth. No width resolves those. Widening 16–24 bits buys a factor of three and then hits a wall.

Solids recorded **4** exact zeros; the torus knot **0**. The two meshes bracket the design space from opposite ends.

Overdraw, measured by accident. The z-buffer is cleared to `+inf`, so the first fragment at each pixel is routed to the non-finite counter — which therefore counts distinct pixels covered. The ratio of total comparisons to it is average overdraw:

mesh	fill rate	overdraw
solids_scene	28.3%	1.350
torus_knot	24.7%	1.375
IronMan	12.4%	4.481

Iron Man’s 12.4% fill means 88% of edge evaluations land on pixels outside the triangle — the argument for hierarchical or edge-walking traversal over a naive bounding-box scan.

3.10 Resolution scaling

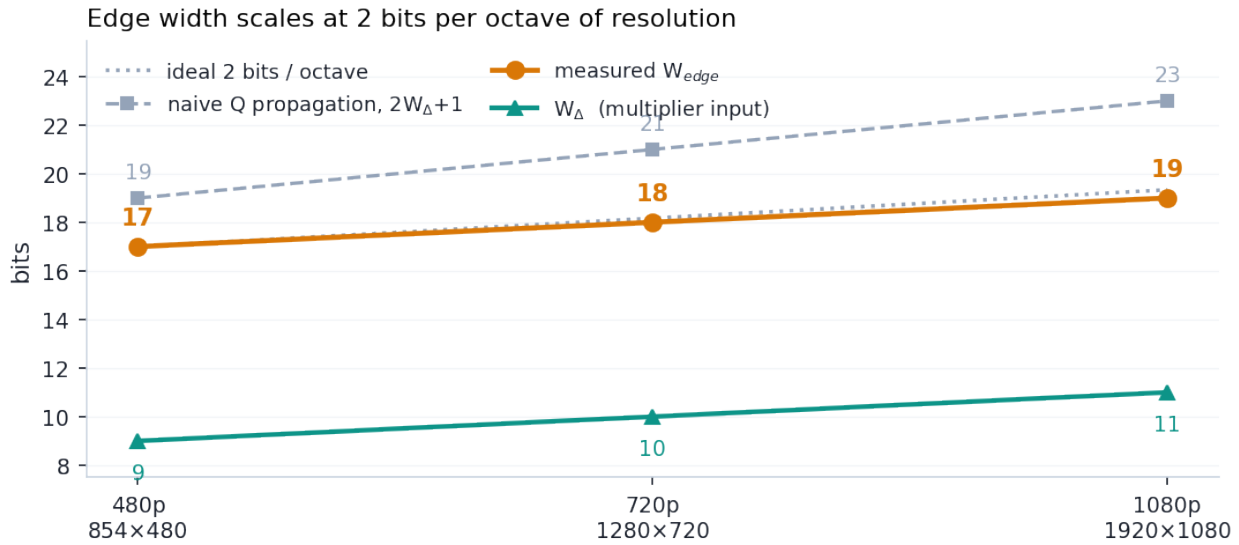


Figure 5: Edge accumulator width against resolution

resolution	width×height	delta bin	edge bin	area bin	screen bin	W_{edge}
480p	854×480	7	15	15	9	17
720p	1280×720	8	16	16	10	18
1080p	1920×1080	9	17	17	10	19

Predicted $\Delta W = 2 \cdot \log_2(\text{scale})$ gives -2.34 and -1.17 relative to 1080p; measured -2 and -1 , both within the ± 1 bin quantisation. The gap between the grey dashed line and the orange one in the figure is the over-provisioning of $\$3.4$, holding across the sweep. Fitting $W = 2 \cdot \log_2(\text{width}) + c$ gives $c = -2.476, -2.644, -2.814$ — **constant to a third of a bit** across a $2.25\times$ range.

FOV, camera distance, model scale and resolution are one knob. All four act through the largest triangle’s screen extent L , and $E_{max} \propto L^2$, so each adds exactly 2 bits per octave. Sweeping them separately re-measures the same relationship; the three meshes already provided the sweep (delta bins 6, 7, 9 — three octaves of L), and the resolution runs confirmed it by a second, independent mechanism.

4. The fractional field: the sub-pixel sweep

The histogram cannot answer this. “How many fractional bits?” is not a range question; it is “how many pixels change coverage when I quantise”, which requires the fixed-point rasterizer to exist and be diffed against the float golden.

Architecture

`drawTriangleFixed` sits **beside** `drawTriangle`, not behind a flag inside it, and selection is a runtime parameter.

Both choices are forced. The sweep needs many values of s from one binary — a `#define` would mean one rebuild per point. More importantly, the golden must be provably byte-identical between a reference run and a sweep run: float codegen is sensitive enough that rebuilding at `-00` instead of `-02` already moves $\sim 135,000$ pixels per frame at 1080p, so a branch inside `drawTriangle` could perturb it.

Only **coverage** is fixed-point: vertex snapping, the edge function, the inside test. Depth, perspective correction and colour blending are copied verbatim and still run in float, so every measured difference is a coverage decision and nothing else.

Vertices are snapped once, in the viewport transform. Two triangles sharing edge AB then read bit-identical integers and compute a bit-identical edge function, so the shared edge is in exactly the same place for both and no crack opens. This part is derivable; the experiment only sets the width.

The metric: read diff ≥ 2

The difference histogram is **two populations**, and only one measures quantisation.

diff = 1 is arithmetic-path noise. 96% of the total at $s=0$ and 100% at $s=16$, and it barely moves across the whole sweep while the coverage signal falls by three orders of magnitude. The fixed path reaches the barycentric weights by a different route than the reference even when the values agree to eight digits, and the truncating `uint8_t` cast turns a last-bit difference into a whole level.

diff ≥ 2 is coverage. A pixel switched between model and background, or between two materials.

Reading the raw total on the solids scene at 1080p would report 1,604,919 $\rightarrow$ 1,386,515: a 14% improvement, suggesting fixed point barely converges. The coverage signal converges to **zero**.

Result

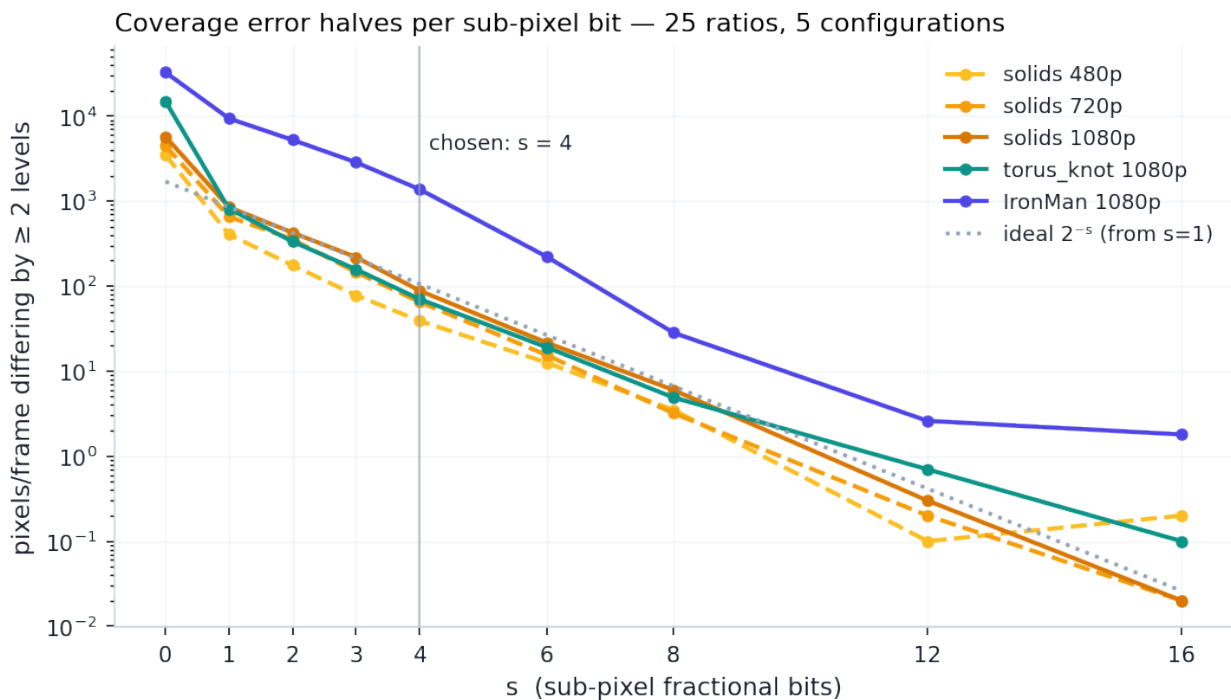


Figure 6: Coverage error against sub-pixel bits for five configurations

(Values of exactly zero are plotted at the axis floor.)

run	triangles	ratios per bit, s=1→8
solids 480p	3,636	2.31, 2.25, 2.00, 1.76, 1.90
solids 720p	3,636	1.84, 2.44, 2.24, 2.05, 2.20
solids 1080p	3,636	2.00, 1.94, 2.48, 2.02, 1.90
torus_knot 1080p	12,800	2.43, 2.12, 2.24, 1.92, 1.97
IronMan 1080p	217,038	1.79, 1.84, 2.08, 2.49, 2.81

25 measured ratios, all bracketing 2, across three resolutions and three meshes spanning 60× in triangle count.

Iron Man's counts sit an order of magnitude above the others at every s, but not by the factor its 60× triangle advantage would suggest. **diff ≥ 2 counts only visible coverage error.** Most of Iron Man's edge length is interior — between triangles of the same material with nearly identical Gouraud shading — and a flip there changes the pixel by one level or less, landing in the diff = 1 population. Only silhouettes and material boundaries produce a ≥2 change, and a mesh with 60× the triangles does not have 60× the silhouette.

Derivation: snapping displaces a vertex by at most $2^{-(s+1)}$ px, so an edge line moves by at most that distance perpendicular, and the band of pixels whose coverage can flip is proportional to (edge length × displacement). Halving the displacement halves the count.

s = 0 sits off the curve by construction. With zero fractional bits there is no representation of half a pixel, so the sample point falls on the pixel *corner* rather than its centre — a systematic half-pixel shift on top of the snapping error. Two error sources where every other s has one. The effect is worst on small triangles: the torus knot's ratio to s=1 is 18.4 against 6.7 for solids, because a 0.5 px displacement is 5% of a 10 px triangle and 2% of a 25 px one.

Convergence at s = 16 — the correctness proof

run	diff ≥ 2	max channel
solids 720p	0	1
solids 1080p	0	1
solids 480p	2	92
torus_knot 1080p	1	60
IronMan 1080p	18	95

Two configurations converge **exactly**: zero pixels differ by more than one level, anywhere, in any frame. Given enough precision the fixed-point rasterizer reproduces the golden model exactly, so every difference at lower s is quantisation and nothing else.

Iron Man's residual is the same coincident geometry zdiff measured earlier, appearing in a second, independent instrument.

The noise floor scales inversely with tessellation density

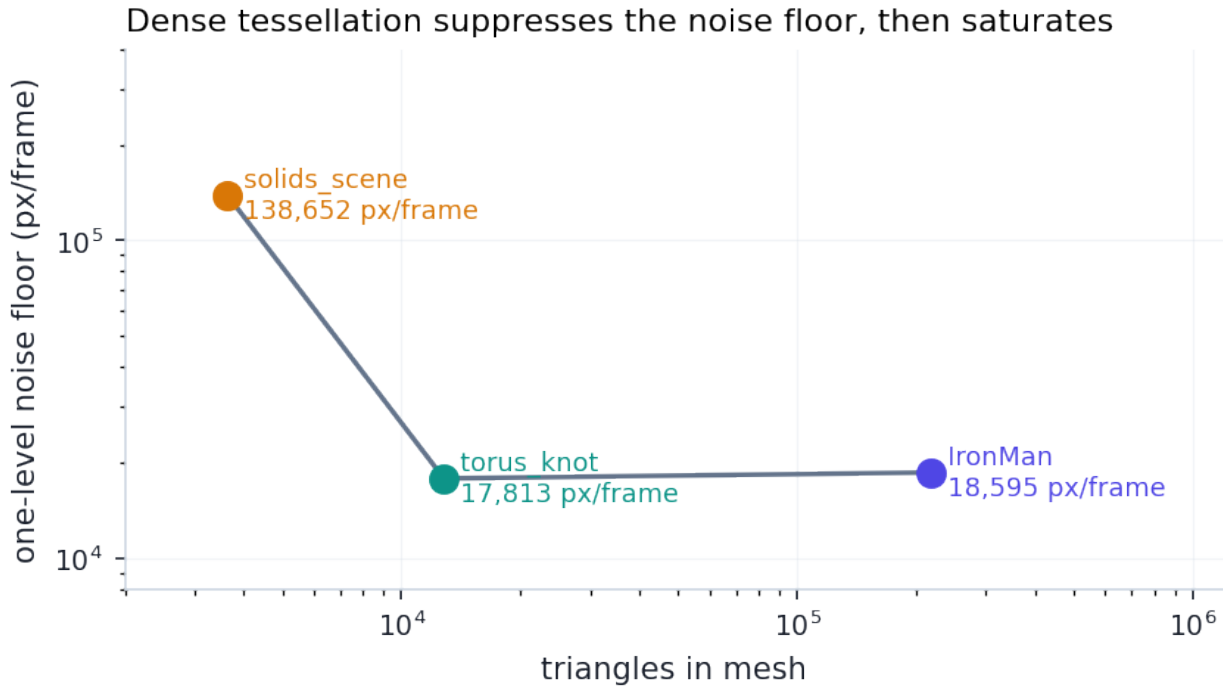


Figure 7: One-level noise floor against triangle count

model	triangles	floor (px/frame)
solids_scene	3,636	138,652
torus_knot	12,800	17,813
IronMan	217,038	18,595

A $3.5\times$ increase in triangle count gives a $7.8\times$ *reduction*; a further $17\times$ increase gives nothing.

Mechanism. Gouraud interpolates between three vertex colours across a triangle. On a 310 px triangle those colours differ noticeably, so a small weight error moves the result across a level. On a 7 px triangle the three vertex normals are nearly parallel, the three colours nearly equal, and the interpolated value barely depends on the weights. Past a certain density the colours are already as close as they can get, and further subdivision buys nothing — hence the saturation.

It also runs the other way with resolution: as a fraction of the screen the floor is 9.79% at 480p, 6.74% at 720p, 6.69% at 1080p. Lower resolution means fewer pixels per triangle for the same colour spread, so a steeper gradient per pixel. **The acceptance threshold is resolution-dependent in the counterintuitive direction.**

5. Conclusion

Integer field: 19 bits measured / 22 bits workload-independent
 Fractional field: $s = 4$, contributing $2s = 8$ bits
 Edge accumulator: 27 bits measured / 30 bits as recommended for build
 Depth field: 0 integer bits + 16 fractional

$\text{diff} \geq 2$ per frame at $s = 4$:

run	px/frame	% of screen
solids 480p	39.1	0.0095%
solids 720p	65.2	0.0071%
solids 1080p	88.5	0.0043%
torus_knot 1080p	70.5	0.0034%
IronMan 1080p	1,379.4	0.066%

$s = 8$ costs eight more bits and forces a 64-bit datapath for a 10–50× reduction in counts already below 0.1% of the screen.

Architectural findings

1. The width equation, not the width. $W_{\text{edge}} = 2 \cdot \log_2(L) + 2$, confirmed by two independent mechanisms across five runs.

2. L is a hardware parameter, enforceable upstream. Splitting oversized triangles in the geometry front end makes width a design decision rather than a measurement. Note this is *not* tessellation, which creates geometry for visual detail; splitting is invisible in the output and purely an implementation concern. The trade — cheap per-triangle work in the narrow stage buying width in the wide per-fragment stage — is structurally identical to backface culling. That is the second instance of the same principle in this pipeline, now measured rather than asserted, and it is why real GPUs bound triangle size in hardware.

3. Coverage and depth scale differently. $W_{\text{edge}} \propto 2 \cdot \log_2(\text{resolution})$; W_{depth} is resolution-invariant to three significant figures across a 4.5× change in fragment count. Coverage is about *where samples fall*; depth is about *how far apart surfaces sit in z* .

4. Depth width has no sufficient value. Unlike the edge function, whose ceiling is geometric and derivable, depth resolution is set by the scene’s minimum surface separation, which is unbounded — coplanar polygons differ by zero. This is why real hardware pairs a 24-bit buffer with a polygon-offset mechanism rather than relying on width alone.

5. Naive Q propagation is systematically pessimistic for geometrically-linked operands: 2–4 bits on this block, and the situation recurs in any dot product, cross product or matrix multiply whose operands come from the same object.

Acceptance criteria for the hardware implementation

A hardware rasterizer diffed against this reference model at $s = 4$, 1080p, should show approximately:

model	diff = 1 /frame	diff ≥ 2 /frame	isolated %
solids_scene	~139,300	~89	~2.8%
torus_knot	~22,400	~71	~19%
IronMan	~24,100	~1,379	~14%

A diff within these bounds is quantisation. A diff above them is a bug. The $\text{diff} = 1$ population is the reference’s own float rounding and is irreducible by adding fractional bits.

The isolated column separates two failure modes a pixel count cannot: scattered single pixels with a large channel difference are depth tie-breaking on near-coplanar geometry, while contiguous runs along triangle edges are coverage shift. A correct build shows the first plus a predictable amount of the second.

6. What transfers, and what does not

quantity	scene-dependent?
2^{-s} slope	no — 25/25 measurements
$W = 2 \cdot \log_2(L) + 2$	no
$\max\ E\ == \ area\ $	no
Barycentric density $2(1-w)$	no
Q propagation over-provisions by 2-4 bits	no
Depth width independent of resolution; edge width scales	no
Width uncorrelated with triangle count	no (the <i>fact</i> ; the magnitude varies)
Depth non-uniformity of 6.35 bits	depends on near/far and camera
Divider denominator ≥ 0.25	depends on camera and near plane
$W = 19$ at 1080p	yes
$F = 16$ for depth	yes

7. Limitations

Temporal error is not measured. Direct3D mandates $s = 8$ not because of static error — 89 versus 6 pixels is invisible in a still — but because at low s an edge snaps between quantised positions as geometry rotates, which reads as crawling along silhouettes. A frame-versus-reference diff cannot see this. Measuring it means diffing *consecutive frames of the same run* at different s , which is a different experiment and has not been done.

Instrument granularity is ± 1 bit per quantity. This is why the Q-propagation slack reads as 2-4 rather than a single value. A running float max alongside the histogram would resolve it.

Not instrumented: the colour path ($c0..c2$ and the uint8 products — low value, since weights are in $[0,1]$ and colour is 8-bit by definition), and the divider *output*, which is derivable from $invw$ without measurement.

Depth and colour quantisation are untested. drawTriangleFixed runs both in float. They are separate experiments with separate answers.

Three meshes are not a workload. They were chosen to bracket a space — coarse/flat, uniform/dense, human-modelled — and they do, but a fourth mesh could still fall outside. That is the argument in §1 for the workload-independent width.

8. Reproduction

```
# Histograms
# stats.h:      #define HISTOGRAM_STATS 1
# main.cpp:     set MODEL_PATH and STATS_PREFIX
# framebuffer.h: set VIEWPORT_WIDTH / VIEWPORT_HEIGHT
mkdir -p stats && make clean && make && ./renderer

# Sub-pixel sweep
# stats.h:      #define HISTOGRAM_STATS 0
make clean && make
```

```
./renderer                                # float golden -> frames/
for s in 0 1 2 3 4 6 8 12 16; do
    ./renderer $s                        # fixed-point -> frames_$$s/
    ./ppmdiff frames frames_$$s 10 > diff_$$s.csv
done
```

The histogram pass writes one CSV per signal, <prefix>_<signal>.csv, columns exponent,count, with zero, clamp_lo, clamp_hi rows appended and empty bins skipped. With HISTOGRAM_STATS 0 the instrument emits no code and the renderer is bit-identical to the uninstrumented build.

ppmdiff writes per-frame CSV to stdout and a summary plus difference histogram to stderr, so the two can be redirected separately.

Sanity check. Every valid run in this study has a ppmdiff mean channel difference between 0.57 and 7.6. **If it exceeds ~10, the two directories were produced by different configurations** — model, resolution or frame count — and the golden needs re-rendering. This failure produced a full sweep of plausible-looking, meaningless numbers before it was caught, which is why the renderer now echoes its full configuration at startup.

Data

file	contents
stats/<model>_<res>_<signal>.csv	histograms: 45 files, nine signals × five configurations
stats/sweep_<model>_<res>_summary.csv	sweep: one row per s
stats/sweep_ALL_RUNS.csv	all five sweeps in one table

Appendix — signal summary, all runs

Format: top bin / bottom bin / total tallies. Histogram pass, 120 frames.

signal	solids 1080p	torus 1080p	IronMan 1080p	solids 720p	solids 480p
edge	17 / -22 / 715,426,158	11 / -22 / 995,171,265	13 / -28 / 2,136,285,247	16 / -19 / 331,301,394	15 / -24 / 156,504,105
area	17 / -10 / 436,320	11 / -13 / 1,536,000	13 / -29 / 26,044,540	16 / -11 / 436,320	15 / -12 / 436,320
delta	9 / -15 / 2,616,328	6 / -15 / 9,215,984	7 / -18 / 156,254,698	8 / -15 / 2,616,328	7 / -16 / 2,616,328
screen	10 / 2 / 472,320	10 / -5 / 1,536,000	10 / -10 / 31,142,160	10 / 1 / 472,320	9 / 1 / 472,320
recw	0 / -2 / 236,160	0 / -2 / 768,000	0 / -2 / 15,571,080	0 / -2 / 236,160	0 / -2 / 236,160
invw	0 / -2 / 55,986,811	0 / -2 / 68,821,685	0 / -2 / 42,317,527	—	—
bary	-1 / -30 / 202,651,317	-1 / -30 / 245,822,916	-1 / -30 / 264,391,623	-1 / -27	-1 / -29
z	-1 / -1 / 67,550,439	-1 / -1 / 81,940,972	-1 / -1 / 88,130,541	-1 / -1 / 30,021,829	-1 / -1 / 13,342,775

signal	solids 1080p	torus 1080p	IronMan 1080p	solids 720p	solids 480p
zdiff	-3 / -23 / 17,513,262	-3 / -19 / 22,332,306	-3 / -24 / 68,292,161	-3 / -23	-3 / -21

Consistency checks

Every per-triangle and per-vertex total matched its predicted value exactly, which validates the instrument before any conclusion is drawn:

check	expected	measured
torus area	12,800 tri $\times$ 120 frames	1,536,000 yes
torus delta	6 $\times$ 1,536,000	9,215,984 + 16 zeros yes
torus screen	6,400 vtx $\times$ 120 $\times$ 2	1,536,000 yes
torus recw	6,400 $\times$ 120	768,000 yes
torus bary	3 $\times$ covered fragments	245,822,916 = 3 $\times$ 81,940,972 yes
torus zdiff	finite + inf = covered	22,332,306 + 59,608,666 = 81,940,972 yes

Sweep data was re-verified on load for all five runs: per-frame counts sum to the reported total, isolated counts sum to the reported total, the difference histogram sums to the total, and its largest occupied bucket equals the reported maximum channel difference.

Miscellaneous counters

- **delta zeros** — axis-aligned edges where Δx or Δy is exactly 0: solids 1,592; torus 16; Iron Man 12,662. Consistent with mesh style — boxes and flat panels have axis-aligned edges, a swept knot has almost none.
- **edge zeros** — a sample point exactly on an edge line: 0, 0 and 2. Two exact hits in 2.1 billion, rare because vertices are irrational after rotation while sample points sit at $k+0.5$ — but nonzero, which is why the $\text{ilogb}(0)$ guard was not paranoia.
- **clamp_lo/clamp_hi** zero everywhere except zdiff, where clamp_hi deliberately counts the +inf first-hits and doubles as the distinct-pixels counter.